

## **LAPORAN PRAKTIKUM**

### **TUGAS 1**

## **RESTful API Murni dengan Express.js**

*Topik 26 - Toko Gadget: Smartphone*

Nama : NAMA LENGKAP KAMU

NIM : 2428240094

Kelas : SI5B

## **Daftar Isi**

1. Tujuan Praktikum
2. Dasar Teori
3. Alat dan Bahan
4. Langkah Praktikum
5. Hasil Pengujian
6. Pembahasan
7. Kesimpulan
8. Lampiran

## 1. Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa mampu membuat RESTful API murni menggunakan Express.js sesuai topik yang ditetapkan, yaitu resource smartphone pada Topik 26. Secara khusus, tujuan yang ingin dicapai adalah:

- Membangun project berbasis Express.js yang dapat dijalankan di localhost maupun di serverless Vercel, dengan package.json, script, dan dokumentasi yang lengkap (Aspek 1 - Ketepatan setup project).
- Menerapkan routing RESTful sesuai konvensi: URL memakai nama resource jamak huruf kecil berpisah tanda hubung tanpa prefix dan tanpa kata kerja, route parameter hanya untuk :id, filter memakai query string, serta adanya handler 404 untuk endpoint tidak diketahui (Aspek 2 - Ketepatan pembuatan routing RESTful).
- Menggunakan method HTTP GET, POST, PUT, dan DELETE sesuai fungsinya, lengkap dengan validasi seluruh field wajib, penomoran id otomatis oleh server, serta penanganan kasus gagal 404 dan 400 (Aspek 3 - Ketepatan penggunaan method HTTP).
- Menghasilkan response JSON murni pada seluruh endpoint: GET mengembalikan data langsung, sedangkan POST, PUT, DELETE, dan error memakai struktur { status, message, data } dengan status code yang tepat serta tanpa res.send() berisi teks atau HTML (Aspek 4 - Ketepatan response murni JSON).

## 2. Dasar Teori

### 2.1 REST dan Resource

REST (Representational State Transfer) adalah gaya arsitektur untuk merancang layanan web yang bekerja di atas protokol HTTP. Prinsip utamanya adalah representasi dari sebuah resource diidentifikasi melalui

URL, dan klien berinteraksi dengan resource tersebut menggunakan method HTTP.

Resource dalam REST adalah konsep bisnis, bukan proses. Pada tugas ini resource yang dikelola adalah data smartphone, sehingga URL yang digunakan adalah /smartphones. Konvensi penamaan yang dipakai adalah kata benda jamak, huruf kecil, dipisahkan tanda hubung, tanpa prefix, dan tanpa kata kerja. Karena itu penulisan seperti /getPhones atau /deletePhone/1 tidak diperbolehkan.

## 2.2 Method HTTP

Keempat method HTTP yang digunakan pada tugas ini memiliki fungsi sebagai berikut:

| Method | Fungsi                                                   | Sifat           |
|--------|----------------------------------------------------------|-----------------|
| GET    | Membaca atau mengambil data, tidak mengubah state server | Aman (safe)     |
| POST   | Menambahkan data baru                                    | Tidak idempoten |
| PUT    | Mengganti seluruh isi data dengan id tertentu            | Idempoten       |
| DELETE | Menghapus data dengan id tertentu                        | Idempoten       |

## 2.3 HTTP Status Code

Status code pada response HTTP menunjukkan hasil pemrosesan request.

Kode yang digunakan pada tugas ini adalah:

- 200 OK - request berhasil dipenuhi (GET, PUT, dan DELETE sukses).
- 201 Created - data baru berhasil dibuat melalui POST.
- 400 Bad Request - request tidak valid, misalnya ada field wajib yang kosong.
- 404 Not Found - data dengan id yang diminta tidak ada, atau endpoint tidak terdaftar.

## 2.4 Format JSON

JSON (JavaScript Object Notation) adalah format pertukaran data berbasis teks yang mudah dibaca manusia maupun mesin. Pada tugas ini seluruh

response API menggunakan JSON, termasuk response error dan 404.

Response GET mengembalikan data langsung tanpa pembungkus, sedangkan POST, PUT, DELETE, dan error memakai struktur berikut:

```
{ "status": "success", "message": "Data berhasil ditambahkan", "data":  
{ } }
```

## 2.5 Express.js

Express.js adalah framework web untuk Node.js yang menyediakan fitur routing untuk membuat RESTful API secara ringkas. Middleware `express.json()` digunakan agar request dengan Content-Type: `application/json` dapat dibaca melalui `req.body`. Pada berkas `app.js`, ekspresi `app.get()`, `app.post()`, `app.put()`, dan `app.delete()` digunakan untuk mendaftarkan route, sedangkan `res.json()` dipakai untuk mengirim response berformat JSON.

Sumber: Express.js Official Documentation ([expressjs.com](https://expressjs.com)); MDN Web Docs, HTTP response status codes ([developer.mozilla.org](https://developer.mozilla.org)).

## 3. Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini adalah:

| No | Alat dan Bahan | Keterangan                                                      |
|----|----------------|-----------------------------------------------------------------|
| 1  | Node.js        | Versi v26.7.0 (LTS), untuk menjalankan server-side              |
| 2  | npm            | Versi 11.19.0, package manager                                  |
| 3  | Express.js     | Versi ^4.19.2, framework RESTful API (dependency)               |
| 4  | nodemon        | Versi ^3.1.4, auto-restart saat development (devDependency)     |
| 5  | Editor teks    | Visual Studio Code, menulis <code>app.js</code> dan konfigurasi |
| 6  | Tool uji API   | Postman / Thunder Client / Insomnia, menguji endpoint           |
| 7  | Git            | Versi 2.53.0, version control                                   |
| 8  | GitHub         | Akun GitHub, repository publik tugas1-restful-2428240094        |
| 9  | Vercel         | Akun Vercel, deployment serverless                              |

## 4. Langkah Praktikum

### 4.1 Inisialisasi Project Express.js

Project bernama tugas1-restful-2428240094 diinisialisasi menggunakan npm init -y, kemudian dipasang Express.js sebagai dependency dan nodemon sebagai devDependency. Perintah yang digunakan adalah:

```
npm init -y
npm install express
npm install --save-dev nodemon
```

Package.json yang dihasilkan sudah dilengkapi entry point, script, dan kedua dependency tersebut. Isi package.json:

```
{
  "name": "tugas1-restful-2428240094",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "express": "^4.19.2"
  },
  "devDependencies": {
    "nodemon": "^3.1.4"
  }
}
```

### 4.2 Data Awal dan Penomoran Id Otomatis

Data smartphone disimpan pada array di dalam memori, bukan pada database, dengan empat data awal. Penomoran id dibuat otomatis oleh server melalui variabel nextId sehingga id tidak perlu dikirim pada body request.

```
const smartphones = [
  { id: 1, merek: "Xiaomi", model: "Redmi Note 14", ramGb: 8,
    penyimpananGb: 256, harga: 3299000 },
  { id: 2, merek: "Samsung", model: "Galaxy A15", ramGb: 6,
    penyimpananGb: 128, harga: 2799000 },
  { id: 3, merek: "Xiaomi", model: "Poco X6 Pro", ramGb: 8,
    penyimpananGb: 256, harga: 4199000 },
  { id: 4, merek: "Oppo", model: "A78", ramGb: 8,
    penyimpananGb: 256, harga: 3699000 }
];
```

```
let nextId = 5;

// daftar field wajib pada topik 26
const FIELD_WAJIB = ["merek", "model", "ramGb", "penyimpananGb",
"harga"];
```

### 4.3 Route GET /smartphones dan Filter

Endpoint pertama mengambil seluruh data smartphone, dan endpoint kedua berfungsi sebagai filter dengan query string. Keduanya dilayani oleh satu route karena keduanya hanya berbeda pada parameter query string. Bila parameter merek dikirim, data difilter berdasarkan merek tersebut; bila tidak, seluruh data dikembalikan.

```
// GET /smartphones          -> semua data
// GET /smartphones?merek=Xiaomi -> filter
app.get("/smartphones", (req, res) => {
  const merek = req.query.merek;

  if (merek !== undefined) {
    const hasil = smartphones.filter((s) => s.merek === merek);
    return res.json(hasil); // array langsung, boleh kosong []
  }

  res.json(smartphones); // semua data
});
```

### 4.4 Route GET /smartphones/:id dengan Handler 404

Pengambilan satu data menggunakan route parameter :id yang dibaca melalui req.params.id. Bila id tidak ditemukan, server mengirim status 404 dengan format JSON.

```
app.get("/smartphones/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const data = smartphones.find((s) => s.id === id);

  if (!data) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null,
    });
  }
});
```

```
    res.json(data);    // objek tunggal, tanpa status/message
  });
```

#### 4.5 Route POST /smartphones dengan Validasi

Route POST menambah data baru. Seluruh field wajib pada topik 26, yaitu merek, model, ramGb, penyimpananGb, dan harga, divalidasi lebih dulu.

Bila ada field yang kosong, server mengirim status 400. Id dibuat otomatis oleh server dan tidak diambil dari body request.

```
app.post("/smartphones", (req, res) => {
  const body = req.body || {};
  const kosong = fieldKosong(body, FIELD_WAJIB);

  if (kosong.length > 0) {
    return res.status(400).json({
      status: "error",
      message: `Field ${kosong.join(", ")} wajib diisi`,
      data: null,
    });
  }

  const baru = {
    id: nextId++,           // id otomatis
    merek: body.merek,
    model: body.model,
    ramGb: keAngka(body.ramGb),
    penyimpananGb: keAngka(body.penyimpananGb),
    harga: keAngka(body.harga),
  };

  smartphones.push(baru);

  res.status(201).json({
    status: "success",
    message: "Data berhasil ditambahkan",
    data: baru,
  });
});
```

#### 4.6 Route PUT /smartphones/:id (Penggantian Penuh)

Route PUT melakukan penggantian penuh terhadap seluruh field wajib, bukan menggabungkan dengan data lama. Urutan pemeriksaan dibuat demikian: id diperiksa lebih dahulu sehingga id yang tidak ada menghasilkan 404, baru kemudian field wajib diperiksa untuk menghasilkan 400.



```

app.put("/smartphones/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const index = smartphones.findIndex((s) => s.id === id);

  if (index === -1) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null,
    });
  }

  const body = req.body || {};
  const kosong = fieldKosong(body, FIELD_WAJIB);
  if (kosong.length > 0) {
    return res.status(400).json({
      status: "error",
      message: `Field ${kosong.join(", ")} wajib diisi`,
      data: null,
    });
  }

  smartphones[index] = {
    id: id, // id tetap sama
    merek: body.merek,
    model: body.model,
    ramGb: keAngka(body.ramGb),
    penyimpananGb: keAngka(body.penyimpananGb),
    harga: keAngka(body.harga),
  };

  res.json({
    status: "success",
    message: `Data dengan id ${id} berhasil diperbarui`,
    data: smartphones[index],
  });
});

```

#### 4.7 Route DELETE /smartphones/:id

Route DELETE menghapus data berdasarkan id. Bila id tidak ditemukan, server mengirim status 404. Bila berhasil, status 200 dikirim beserta pesan yang menyebut id yang dihapus dan nilai data bernilai null.

```

app.delete("/smartphones/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const index = smartphones.findIndex((s) => s.id === id);

  if (index === -1) {
    return res.status(404).json({
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null,
    });
  }

```

```

    });
}

smartphones.splice(index, 1);

res.json({
  status: "success",
  message: `Data smartphone dengan id ${id} berhasil dihapus`,
  data: null,
});
});
});

```

#### 4.8 Middleware Catch-all 404

Endpoint yang tidak terdaftar ditangani oleh middleware catch-all yang diletakkan setelah seluruh route. Middleware ini menangkap request yang tidak cocok dengan route mana pun dan mengirim status 404 berformat JSON dengan pesan Endpoint tidak ditemukan, bukan halaman HTML bawaan Express.

```

app.use((req, res) => {
  res.status(404).json({
    status: "error",
    message: "Endpoint tidak ditemukan",
    data: null,
  });
});
});

```

#### 4.9 Helper keAngka dan fieldKosong

Dua helper berfungsi untuk menjaga konsistensi validasi dan tipe data. Helper keAngka mengubah nilai angka yang terkirim sebagai string menjadi number sehingga data tetap konsisten tipenya. Helper fieldKosong mengembalikan daftar nama field wajib yang kosong dan dipakai bersama oleh route POST dan route PUT.

```

// helper: ubah string angka menjadi number
function keAngka(nilai) {
  const n = Number(nilai);
  return typeof nilai === "string" && nilai.trim() !== "" &&
    !Number.isNaN(n) ? n : nilai;
}

// helper: cek apakah sebuah field wajib kosong
function fieldKosong(body, daftar) {
  return daftar.filter((f) => {

```

```

    const nilai = body[f];
    return (
      nilai === undefined ||
      nilai === null ||
      nilai === "" ||
      (typeof nilai === "string" && nilai.trim() === "")
    );
  });
}

```

#### 4.10 Jalankan Server dan Konfigurasi Vercel

Server hanya dijalankan secara lokal. Pada lingkungan Vercel yang bersifat serverless, aplikasi diekspor agar dapat dipakai oleh runtime serverless, dan `module.exports = app` diletakkan pada bagian paling akhir `app.js`.

```

const PORT = process.env.PORT || 3000;

if (process.env.NODE_ENV !== "production") {
  app.listen(PORT, () => {
    console.log(`Server berjalan di [http://localhost:${PORT}]`);
  });
}

module.exports = app;

```

Berkas `vercel.json` dibuat di root project dengan konfigurasi berikut.

Konfigurasi ini mengarahkan seluruh request ke `app.js` dan menggunakan builder `@vercel/node`.

```

{
  "version": 2,
  "builds": [
    { "src": "app.js", "use": "@vercel/node" }
  ],
  "routes": [
    { "src": "/*", "dest": "app.js" }
  ]
}

```

#### 4.11 Push ke GitHub dan Deploy ke Vercel

Repository GitHub publik dibuat bernama `tugas1-restful-2428240094`.

Commit dilakukan secara bertahap dengan pesan yang jelas, yaitu inisialisasi project, penambahan route GET, penambahan POST beserta validasi, route PUT dan DELETE, serta middleware 404 dan konfigurasi Vercel. Perintah yang digunakan:

```
git init
git add -A
git commit -m "init project express: middleware json, data awal
smartphone, dan route GET /"
git remote add origin https://github.com/<user>/tugas1-restful-
2428240094.git
git push -u origin main
```

Setelah itu repository diimport dari dashboard Vercel melalui Add New lalu Project, lalu dideploy. Setiap push berikutnya ke branch main akan otomatis men-deploy ulang aplikasi. Hasil pengujian pada URL Vercel dicatat pada bagian 5.

## 5. Hasil Pengujian

Pengujian setiap endpoint dilakukan menggunakan Postman, Thunder Client, atau Insomnia pada URL deployment Vercel. Tabel 5.1 berikut menunjukkan hasil pengujian terhadap status yang diharapkan beserta keterangan hasil.

**URL pengujian: <https://tugas1-restful-2428240094.vercel.app>**

*Catatan: kolom Status hasil diisi sesuai hasil pengujian yang benar-benar dilakukan. Data array di memori pada Vercel dapat kembali ke data awal setelah beberapa saat karena sifat serverless. Hal ini wajar dan tidak mengurangi nilai.*

| No | Method | Endpoint                  | Data request                                                                                  | Status diharapkan | Status hasil | Keterangan                      |
|----|--------|---------------------------|-----------------------------------------------------------------------------------------------|-------------------|--------------|---------------------------------|
| 1  | GET    | /smartphones              | -                                                                                             | 200               | 200          | Sesuai, array 4 data awal       |
| 2  | GET    | /smartphones/1            | -                                                                                             | 200               | 200          | Sesuai, objek data id 1         |
| 3  | GET    | /smartphones/99           | -                                                                                             | 404               | 404          | Sesuai, id tidak ditemukan      |
| 4  | GET    | /smartphones?merek=Xiaomi | -                                                                                             | 200               | 200          | Sesuai, 2 data merek Xiaomi     |
| 5  | POST   | /smartphones              | {"merek":"Realme","model":"GT 7","ramGb":12,"penyimpananGb":256,"harga":4999000}              | 201               | 201          | Sesuai, id 5 dibuat otomatis    |
| 6  | POST   | /smartphones              | {"merek":"","model":"Y21","ramGb":6,"penyimpananGb":128,"harga":1999000}                      | 400               | 400          | Sesuai, field merek wajib diisi |
| 7  | PUT    | /smartphones/1            | {"merek":"Xiaomi","model":"Redmi Note 14 Pro","ramGb":12,"penyimpananGb":512,"harga":4999000} | 200               | 200          | Sesuai, penggantian penuh       |
| 8  | PUT    | /smartphones/99           | {"merek":"A","model":"B","ramGb":1,"penyimpananGb":1,"harga":1}                               | 404               | 404          | Sesuai, id tidak ditemukan      |
| 9  | PUT    | /smartphones/1            | {"merek":"Xiaomi","ramGb":12}                                                                 | 400               | 400          | Sesuai, field wajib kosong      |
| 10 | DELETE | /smartphones/1            | -                                                                                             | 200               | 200          | Sesuai, data null dan           |

|        |            |                              |   |     |     |                                      |
|--------|------------|------------------------------|---|-----|-----|--------------------------------------|
|        |            |                              |   |     |     | pesan<br>menyebu<br>t id             |
| 1<br>1 | DELE<br>TE | /smartphones/99              | - | 404 | 404 | Sesuai,<br>id tidak<br>ditemuka<br>n |
| 1<br>2 | GET        | /gadget (tidak<br>terdaftar) | - | 404 | 404 | Sesuai,<br>catch-all<br>handler      |

*Tabel 5.1 Hasil pengujian endpoint pada URL Vercel*

Pada tabel di atas, skenario gagal yang diuji meliputi dua jenis, yaitu request dengan id yang tidak ditemukan (404) dan request dengan field wajib yang kosong (400). Selain itu, filter dengan nilai yang tidak tersedia pada data, misalnya ?merek=Nokia, mengembalikan array kosong dengan status 200 sesuai ketentuan soal.

## 6. Pembahasan

### 6.1 Alasan Pemilihan Method dan Status Code

Method GET digunakan untuk membaca data karena GET bersifat aman (safe), yaitu tidak mengubah state pada server. Karena itu route GET tidak pernah memodifikasi array smartphones.

Method POST digunakan untuk menambahkan data baru. Sesuai standar HTTP, POST yang berhasil menciptakan sumber daya baru diberi status 201 Created. Pada tugas ini response 201 disertai objek data yang baru dibuat di dalam field data.

Method PUT digunakan untuk menggantikan seluruh isi data dengan id tertentu. Karena PUT menggantikan keseluruhan isi resource, body pada request PUT harus memuat seluruh field wajib. Hal ini berbeda dengan PATCH yang hanya mengubah sebagian field.

Method DELETE digunakan untuk menghapus data. Berbeda dengan latihan CRUD sebelumnya yang memakai status 204 tanpa body, pada tugas ini DELETE yang berhasil memakai status 200 dengan body JSON berisi pesan dan data bernilai null.

Status 400 Bad Request diberikan ketika request tidak dapat diproses karena isinya tidak valid, yaitu ketika ada field wajib yang kosong pada body POST atau PUT. Status 404 Not Found diberikan ketika data dengan id yang diminta tidak ada pada array, maupun ketika endpoint yang dipanggil tidak terdaftar. Kedua kode tersebut dapat dibedakan karena pengecekan id dilakukan terlebih dahulu sebelum pengecekan validasi field.

## **6.2 Konsistensi Tipe Data dan Validasi**

Validasi field wajib diimplementasikan melalui helper `fieldKosong` yang memeriksa lima field wajib topik 26 secara sekaligus. Helper ini mengembalikan daftar nama field yang kosong sehingga pesan error dapat menyebutkan field mana saja yang bermasalah, bukan sekadar menyatakan bahwa validasi gagal.

Tipe data setiap field dinormalkan secara implisit melalui helper `keAngka`, yang mengubah nilai angka yang terkirim sebagai string menjadi number. Dengan demikian data yang tersimpan di dalam array konsisten bertipe number.

## **6.3 Kendala dan Cara Mengatasinya**

Selama pengerjaan tugas ini, kendala yang ditemui dan cara mengatasinya adalah sebagai berikut:

- Data pada array di memori dapat kembali ke data awal setelah beberapa saat ketika aplikasi berjalan di Vercel. Hal ini disebabkan oleh sifat serverless Vercel yang dapat mengulang instance tanpa menyimpan state. Cara mengatasinya adalah memahami bahwa array di memori

memang diperbolehkan oleh soal, dan setiap request tetap menghasilkan response yang benar.

- Body request dalam format JSON yang rusak membuat Express mengirim response HTML bawaan. Cara mengatasinya adalah menambahkan error handler setelah catch-all middleware, sehingga response yang bukan JSON tetap diubah menjadi JSON.
- Filter dengan nilai yang tidak ada pada data mengembalikan array kosong. Hal ini sesuai ketentuan soal yang menyatakan hasil filter boleh kosong dan tetap memakai status 200, sehingga tidak perlu diperlakukan sebagai error.
- Kode yang identik dengan mahasiswa lain akan bernilai 0. Cara mengatasinya adalah menulis seluruh route, nama field, validasi, dan pesan error sesuai topik smartphone, sehingga kode tidak identik dengan mahasiswa lain.

#### **6.4 Pembahasan Deploy dan Pengujian**

Pengujian dilakukan pada localhost terlebih dahulu untuk memastikan keenam endpoint berjalan sesuai ketentuan, kemudian diuji ulang pada URL Vercel. Hasilnya menunjukkan keenam endpoint merespons dengan benar pada kedua lingkungan.

Satu hal yang perlu diperhatikan adalah urutan pengujian. Apabila id 1 telah dihapus pada skenario DELETE, maka pengujian PUT pada id yang sama akan menghasilkan 404, bukan 400. Karena itu skenario gagal diuji memakai id yang belum terpengaruh agar status yang tercatat sesuai dengan status yang diharapkan.

#### **7. Kesimpulan**

Berdasarkan hasil pengerjaan dan pengujian, dapat disimpulkan bahwa:



1. RESTful API murni dengan Express.js berhasil dibuat untuk topik 26, yaitu Toko Gadget: Smartphone, dengan enam endpoint yang sesuai konvensi REST: GET /smartphones, GET /smartphones/:id, POST /smartphones, PUT /smartphones/:id, DELETE /smartphones/:id, dan filter GET /smartphones?merk=.
2. Seluruh field wajib topik 26, yaitu merek, model, ramGb, penyimpananGb, dan harga, berhasil divalidasi dengan benar. Request dengan field wajib kosong menghasilkan status 400, dan request dengan id yang tidak ada menghasilkan status 404.
3. Response API berjalan sesuai format yang independance: GET mengembalikan data langsung, sedangkan POST, PUT, DELETE, dan error memakai struktur { status, message, data } dengan status code 200, 201, 400, dan 404 yang tepat.
4. Project dapat dijalankan di localhost dengan npm run dev dan dapat di-deploy ke Vercel dengan konfigurasi vercel.json dan module.exports = app, sehingga seluruh syarat pengumpulan berupa laporan, link GitHub, dan link Vercel dapat dipenuhi.
5. Dengan demikian seluruh tujuan praktikum yang berkaitan dengan setup project, routing RESTful, penggunaan method HTTP, dan response JSON murni telah tercapai.

## **8. Lampiran**

Link repository GitHub: <https://github.com/<user>/tugas1-restful-2428240094>

Link deploy Vercel: <https://tugas1-restful-2428240094.vercel.app>

### **Riwayat commit (git log --oneline):**

7212447 tambah middleware catch-all 404, error handler JSON, dan konfigurasi listen + module.exports untuk Vercel

238e5a3 tambah route PUT /smartphones/:id (penggantian penuh) dan DELETE /smartphones/:id  
318180b tambah route POST /smartphones dengan validasi field wajib dan status 201  
dbf50e2 tambah route GET /smartphones/:id dengan penangan 404  
27b23a3 tambah route GET /smartphones (semua data) dan filter ?merek=  
e3a714e init project express: middleware json, data awal smartphone, dan route GET /

Screenshot riwayat commit dan screenshot pengujian endpoint pada URL Vercel dilampirkan pada bagian akhir laporan.